

Non-Aligned Movement Imperialist Competitive Algorithm (NAM-ICA)

Enhancing the Imperialist Competitive Algorithm (ICA) with Regional Competition

ICA with just one rule changed: which empires are allowed to compete: Early on an empire competes only with its nearest rivals, and that neighborhood widens to global by the end.

1. Introduction and aim

Optimization problems appear in nearly every applied science and engineering domain. When the objective function is non-convex, non-differentiable, or expensive to evaluate, classical gradient-based methods often fail. Metaheuristic algorithms address this by treating the objective as a black box and searching the solution space using stochastic, often, population-based strategies inspired by natural or social processes.

The Imperialist Competitive Algorithm (ICA), introduced by Atashpaz-Gargari and Lucas in 2007, is one such metaheuristic. Unlike algorithms inspired by biology (particle swarms, ant colonies, genetic evolution), ICA draws its analogy from human socio-political dynamics: candidate solutions form competing empires that grow, shrink, merge, and collapse over time. The algorithm has been successfully applied to engineering design, scheduling, control systems, and feature selection problems.

This project implements the original ICA faithfully in Python, benchmarks it on seven standard CEC test functions, and introduces a modification that improves convergence.

ICA has a known weakness. It commits to one region of the search space too early. Once the strongest group takes the lead, the rest of the search drifts toward it, even when a better region was never checked. This early lock-in has a name: premature convergence, which is when the search stops spreading out and settles before it has looked around enough.

The change NAM-ICA brings is small and structural. The framing is borrowed from the Cold War. During the Non-Aligned Movement, smaller countries refused to be pulled into either the American or the Soviet bloc, which kept more independent players on the board. NAM-ICA does the same thing to empires. Early in the run a distant strong empire is not allowed to swallow a small far-off one. The far-off empires are left alone to search their own corners. The blocs only form near the end, once it is time to commit.

In code terms this is one rule: at first an empire can only take colonies from its nearest neighbours, and the allowed range grows over the run until it is global again.

The aim was to improve ICA on the functions where it reliably gets stuck. These are the deceptive many-valley functions F6 Griewank and F7 Ackley. A many-valley function is one with many false low points that can trap the search short of the true lowest point. The prediction was that both F6 and F7 would improve.

The overall result is modest: on most functions, except one, the change made little difference.

F7 Ackley showed a large median improvement, from 1.23 to 0.42, but the paired Wilcoxon test narrowly missed conventional significance at $p = 0.052$.

F6 Griewank did not move at all.

One more result turned up that was not planned: at a higher randomness setting NAM-ICA was much steadier than plain ICA on F4 HGBat, and that difference was statistically significant. That HGBat result is a secondary observation found after seeing the numbers, not an original goal.

2. The original algorithm: ICA

ICA copies the idea of empires competing for colonies. It maintains a population of candidate solutions, called countries, in the continuous search space. Each country has a position (a real-valued vector) and a cost (the value of the objective function at that position). The algorithm proceeds by organizing countries into empires and then evolving those empires through five operations applied in each iteration (called a decade).

Initialization

A population of $n_{\text{countries}}$ random points is generated within the search bounds. Each point is evaluated and the population is sorted by cost. The best $n_{\text{imperialists}}$ countries become imperialists; the remaining countries become colonies. Each imperialist receives a number of colonies proportional to its relative power (lower cost equals higher power). The original MATLAB code uses a linear power formula: $\text{power} = 1.3 * \text{max_cost} - \text{imperialist_cost}$ when costs are positive, and $\text{power} = 0.7 * \text{max_cost} - \text{imperialist_cost}$ when costs are negative.

2.1 Assimilation

Each colony is pulled toward its imperialist by a random fraction of the vector connecting them. The position update is: $\text{new_position} = \text{old_position} + 2 * \text{beta} * \text{rand} * (\text{imperialist_position} - \text{colony_position})$, where beta is the assimilation coefficient and rand is a uniform random number in $[0, 1]$ drawn independently for each dimension. The factor of 2 allows colonies to occasionally overshoot the imperialist, which encourages exploration beyond the segment between them. Positions are clamped to the search bounds after each update.

2.2 Revolution

A fraction of colonies (controlled by the revolution rate) are replaced with brand new random countries. This is the primary exploration mechanism: it injects fresh diversity and prevents premature convergence. In the original implementation, the revolution rate decays each iteration by a damping factor of 0.99. As a result, early iterations contain heavy exploration while later iterations focus on refinement. Revolution affects only colonies; the imperialist is left untouched.

2.3 Possession (intra-empire competition)

After assimilation and revolution, the best colony in each empire is compared to its imperialist. If the colony has a lower cost, the two swap roles: the colony becomes the new imperialist and the previous imperialist becomes a colony. This ensures that the best solution in each empire always leads it, without disrupting the overall structure.

2.4 Uniting Similar Empires

Two empires whose imperialists lie close to each other in search space are merged. The Euclidean distance between imperialists is compared against a threshold equal to the uniting threshold (0.02 by default) multiplied by the diagonal length of the search space. When two imperialists fall within this threshold, the weaker empire is absorbed by the stronger one: the weak imperialist becomes a colony, along with all its previous colonies, of the stronger empire. This phase prevents redundant empires from converging on the same optimum.

2.5 Imperialistic Competition

With a fixed probability of 0.11 per iteration, the algorithm performs an inter-empire competition. The total cost of each empire is computed as $\text{total_cost} = \text{imperialist_cost} + \text{zeta} * \text{mean}(\text{colony_costs})$, where zeta weighs the contribution of colonies to the empire's strength. The empire with the highest total cost is identified as the weakest. Its worst colony is taken and given to another empire, with the receiving empire chosen probabilistically based on inverse total cost. If the weakest empire ends up with no colonies, it collapses: its imperialist becomes a colony of the empire that won the last competition.

Termination

The algorithm terminates after a fixed number of iterations or earlier if only one empire remains (meaning all countries have been absorbed under a single imperialist). The position of the surviving imperialist is returned as the best solution found.

2.8 Parameters

All parameter values are summarized below:

Parameter:	Value:	Meaning:
n_countries	300	total population size
n_imperialists	20	starting number of empires
n_iterations	2000	max decades (loop iterations)
beta	2.0	how far colony moves toward imperialist: assimilation
rev_rate	0.3	fraction of colonies teleported to random new position
damp_ratio	0.99	how fast the revolution rate shrinks each round
zeta	0.02	colony weight in empire total cost (0.02 is negligible)
uniting_threshold	0.02	merge empires within 2% of search space
competition_prob	0.11	chance the competition step fires at each round: 11%

3. Description and Rationale for the Modification

Plain ICA lets any empire receive the contested colony from the very first round, including a strong empire far across the space. So one strong empire can absorb distant rivals before they have searched their own areas, and the whole search collapses onto one region early. This early lock-in is called premature convergence: the search settles before it has looked around enough. That is the behaviour NAM-ICA targets.

Two terms describe the balance. Exploration means searching wide for new regions. Exploitation means refining inside a region already found. A good optimizer needs both, and premature convergence is exploitation taking over too soon.

4. The benchmark functions

All benchmark functions are generalized to n dimensions and tested as 10 dimensional vectors. Two-variable versions are too easy to solve and not like real problems, which have many variables. As the number of variables grows the space grows fast and premature convergence hurts more, that is why each function is extended to 10 variables.

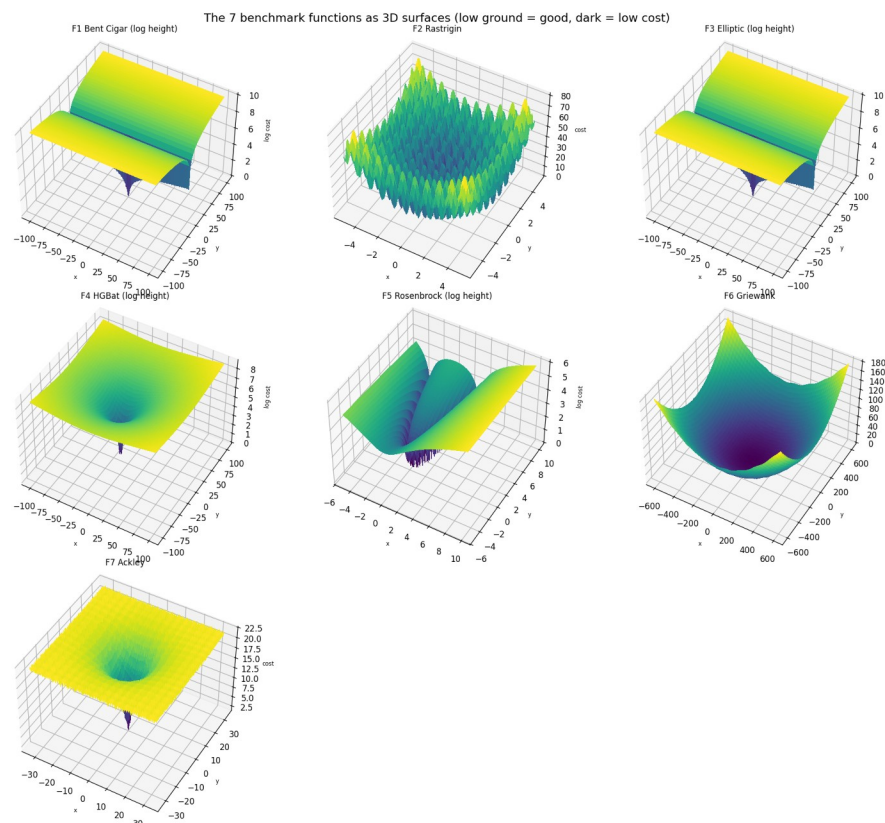


Figure 1. Visual simplification. The seven functions drawn as 2-variable surfaces. Low ground is a good answer. Some heights are shown on a log scale so the huge functions stay readable.

The table below describes each function. Each one has a known lowest value of 0.

Table 1. The seven benchmark functions as landscapes.

Function	Landscape, and why it is easy or hard
F1 Bent Cigar	A single valley that is extremely lopsided: one direction is normal and the rest are vastly steeper. Easy to find the valley, hard to tune to the exact bottom.
F2 Rastrigin	A round bowl covered in a regular grid of bumps. The bowl still steers the search to the centre, so it is solved easily here despite the bumps.
F3 Elliptic	The same lopsided single valley as F1 in two dimensions. They only differ in higher dimensions, where F3 spreads the steepness across more directions and makes precise tuning even harder.
F4 HGBat	A smooth bowl whose lowest point is shifted off-centre, so the bottom is found away from the middle.
F5 Rosenbrock	A long curved valley. Easy to enter, hard to follow down to the bottom.

F6 Griewank	A wide smooth bowl with tiny ripples on it. At full scale the bowl pulls the search to the centre, so it looks deceptively simple. NAM-ICA target
F7 Ackley	A wide flat plain with one deep narrow hole in the middle. The plain gives almost no hint of which way the hole is, which is what makes it hard. NAM-ICA target

Going from two variables to ten is done by applying each function's pattern to all ten. For most functions that means summing the same per-variable term over all ten axes. Rastrigin adds its bump term once for each axis. Ackley averages its terms over all ten. HGBat is written with two shift values in two dimensions, minus ten and minus five, so those two values are alternated across the ten axes. The shape and the difficulty of each function carry over, just in a bigger space.

The targets for the change were the functions where plain ICA reliably got stuck: F6 and F7. The others were not targets. F1 and F2 are solved easily. F3 stays hard because of its skewed shape, not because of false dips, so it is a different kind of problem. F5 is a single curved valley.

5. The modification: NAM-ICA

NAM-ICA changes one rule, in step 5 only. Nothing else about ICA changes.

Using distances between imperialists, NAM-ICA restricts empire competition locally early, then gradually restores global competition.

In ICA the contested colony can go to any empire. In NAM-ICA it can only go to one of the weakest empire's k nearest empires, measured by straight-line distance between imperialists. Among those neighbours the stronger ones are still more likely to win it.

The number k starts at 2 and grows in a straight line over the run, until it covers all empires by the final round. So competition is local early and global late, and the run still ends with one winner, like plain ICA. The point is to stop the search collapsing onto one region too early.

Why this should help? Early lock-in happens because one strong empire absorbs distant rivals before they can explore. If distant rivals are off-limits early, they survive longer and keep searching their own regions, which protects exploration when it matters most, at the start. Letting k grow then shifts the balance back to exploitation, so the search still settles. The change delays the collapse onto one region without preventing it.

This is one structural change with zero new tunable parameters.

The growth from local to global is a fixed automatic rule, not a knob to tune. The only constant is the starting neighbourhood of 2, the smallest that allows any competition at all, which is a design choice and not a tuned value.

In island-model terms the change turns ICA's fully connected competition into a distance-based neighbourhood that grows to fully connected, which is a known type of structure in island-model optimizers.

6. Experimental setup and fairness

Every version uses the same settings: 10 dimensions, 30 runs per function, 2000 iterations, 300 countries, 20 imperialists, and the standard ICA values for the rest. The seeds are 1 to 30, and the same seeds are used for every version, so run 5 of one version starts from the same random state as run 5 of another. The comparison is therefore paired, run by run. Thirty runs are used because the algorithm is random, so one run can be lucky or unlucky.

The comparison is fair by construction. With the modification switched off, the code reproduces plain ICA exactly, run for run, with identical numbers, so the two versions differ only in the one rule. The rule only measures distances between imperialists and never calls the cost function, so both versions spend the same number of cost-function evaluations per round. The budget is equal.

Four versions are compared. The revolution rate is the one setting that is varied, because raising it is the other obvious way to add exploration, by shaking harder. Comparing against a higher revolution rate checks whether NAM-ICA does something that plain extra randomness does not.

- ICA at revolution 0.3, the baseline.
- ICA at revolution 0.5, the baseline with more randomness.
- NAM-ICA at revolution 0.3.
- NAM-ICA at revolution 0.5.

The versions are judged three ways. Accuracy is how low the final cost gets, reported as the median of the 30 runs. The median is the middle value, which is less swayed by one extreme run than the mean. Convergence speed is how fast the cost drops, read from the convergence plots. Robustness is how consistent the runs are, seen in the spread between the best and worst run.

To check whether a difference is real and not luck, the Wilcoxon signed-rank test is used. This is a standard paired test that asks whether one version is reliably better than another across the 30 matched runs. A p-value below 0.05 is the usual line for calling a result statistically significant, meaning unlikely to be chance.

7. Results

Table 2 is the full results table: the lowest, highest, mean, median and standard deviation of the final cost over the 30 runs, for all seven functions and all four versions. Lower is better, and the known minimum is 0. The two shaded medians are worth attention.

Table 2. Final cost over 30 runs, all four versions. Shaded medians mark the main improvement (F7 NAM-ICA $r=0.3$) and the secondary significant result (F4 NAM-ICA $r=0.5$).

Function	Version	min	max	mean	median	std
F1 Bent Cigar	ICA $r=0.3$	1.124e-25	1.426e-6	4.756e-8	1.795e-19	2.560e-7
F1 Bent Cigar	ICA $r=0.5$	1.648e-25	8365.96	278.865	3.267e-19	1501.74
F1 Bent Cigar	NAM-ICA $r=0.3$	6.277e-30	9784.57	924.956	3.175e-20	2778.21
F1 Bent Cigar	NAM-ICA $r=0.5$	3.679e-30	9783.54	326.118	7.596e-19	1756.2
F2 Rastrigin	ICA $r=0.3$	0	2.785e-12	1.118e-13	0	5.023e-13
F2 Rastrigin	ICA $r=0.5$	0	1.596e-10	6.076e-12	0	2.866e-11
F2 Rastrigin	NAM-ICA $r=0.3$	0	2.359e-12	1.620e-13	0	5.729e-13
F2 Rastrigin	NAM-ICA $r=0.5$	0	8.063e-11	3.016e-12	0	1.445e-11
F3 High Conditioned Elliptic	ICA $r=0.3$	1.742e-5	271859	69085.2	56415.9	88724.4
F3 High Conditioned Elliptic	ICA $r=0.5$	0.00129838	271859	77938.3	56415.9	92371.2
F3 High Conditioned Elliptic	NAM-ICA $r=0.3$	1.095e-20	271859	81485.7	56415.9	89663.6
F3 High Conditioned Elliptic	NAM-ICA $r=0.5$	2.294e-19	271859	76721	46415.9	93446.9
F4 HGBat	ICA $r=0.3$	8.004e-6	12.6427	1.51423	0.143248	3.03073
F4 HGBat	ICA $r=0.5$	1.481e-9	108.694	7.73923	1.2521	21.9974
F4 HGBat	NAM-ICA $r=0.3$	4.455e-10	61.6247	4.45903	0.256252	13.2685
F4 HGBat	NAM-ICA $r=0.5$	2.626e-8	157.561	6.40741	0.01614	28.2382
F5 Rosenbrock	ICA $r=0.3$	1.137e-5	3.28928	0.358087	0.0668389	0.806655
F5 Rosenbrock	ICA $r=0.5$	9.012e-4	2.53574	0.370872	0.138321	0.557862
F5 Rosenbrock	NAM-ICA $r=0.3$	4.119e-6	1.25014	0.213528	0.129119	0.309032
F5 Rosenbrock	NAM-ICA $r=0.5$	0.00178713	1.95237	0.325141	0.151804	0.484677
F6 Griewank	ICA $r=0.3$	0.13656	1.02216	0.628584	0.655142	0.257141
F6 Griewank	ICA $r=0.5$	0.100035	1.04153	0.585783	0.55188	0.27747
F6 Griewank	NAM-ICA $r=0.3$	0.103143	1.14325	0.636887	0.664881	0.265568
F6 Griewank	NAM-ICA $r=0.5$	0.100011	1.02978	0.600583	0.60943	0.296837
F7 Ackley	ICA $r=0.3$	0.0847249	2.8077	1.17367	1.23133	0.764389
F7 Ackley	ICA $r=0.5$	0.0109616	2.87281	1.017	0.902826	0.808173
F7 Ackley	NAM-ICA $r=0.3$	0.00922311	2.26914	0.750649	0.424387	0.692308
F7 Ackley	NAM-ICA $r=0.5$	0.0404766	2.93434	1.0613	0.98183	0.783912

Table 3 gives the Wilcoxon test for every function, comparing NAM-ICA against ICA at the same revolution rate. The shaded rows are the one significant result and the one near miss.

Table 3. Wilcoxon signed-rank test, NAM-ICA versus ICA at the matching revolution rate.

Function	Rate	p-value	Lower median	Significant
F1 Bent Cigar	0.3	0.670	NAM-ICA	no
F1 Bent Cigar	0.5	0.855	ICA	no
F2 Rastrigin	0.3	0.752	tie	no
F2 Rastrigin	0.5	0.413	tie	no
F3 Elliptic	0.3	0.675	tie	no
F3 Elliptic	0.5	0.584	NAM-ICA	no
F4 HGBat	0.3	0.440	ICA	no
F4 HGBat	0.5	0.013	NAM-ICA	yes (secondary)
F5 Rosenbrock	0.3	0.777	ICA	no
F5 Rosenbrock	0.5	0.824	ICA	no
F6 Griewank	0.3	1.000	ICA	no
F6 Griewank	0.5	0.968	ICA	no
F7 Ackley	0.3	0.052	NAM-ICA	no (near miss)
F7 Ackley	0.5	1.000	ICA	no

Figure 2 is the main result: F7 Ackley at revolution 0.3, plain ICA against NAM-ICA. The lines are the median run and the bands are the full range across the 30 runs.

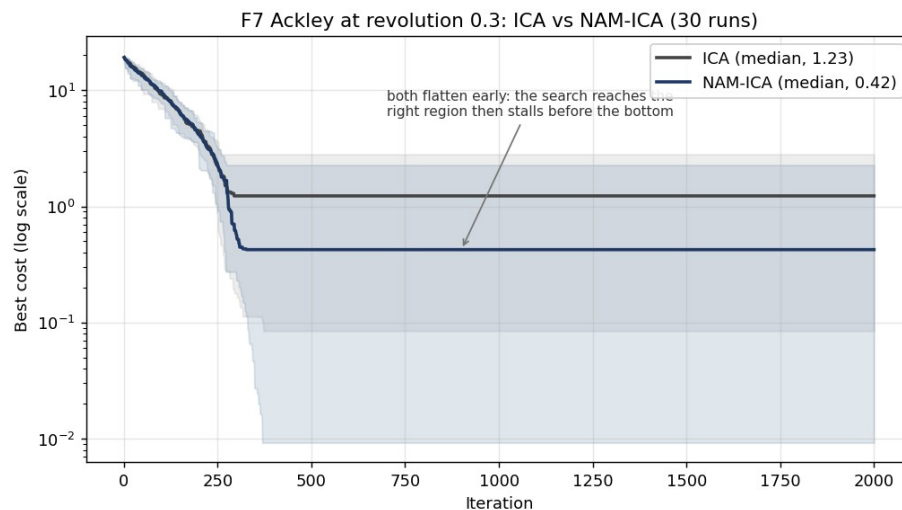


Figure 2. F7 Ackley at revolution 0.3. NAM-ICA settles at a lower typical cost than ICA. Both flatten near iteration 300, which means the search reached the right region and then stalled.

For context, Figures 3 and 4 show the convergence at revolution 0.3 for ICA and NAM-ICA. Each small panel is one function. The red line is the median run, the shaded band is the range from the best to the worst run, and the y-axis is logarithmic so improvement across many orders of magnitude stays visible.

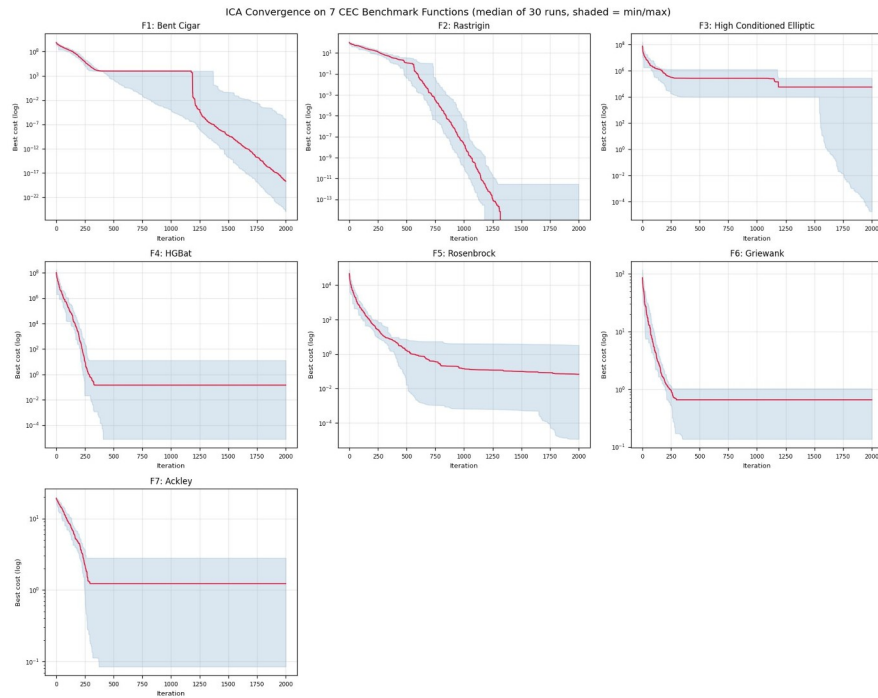


Figure 3. ICA convergence at revolution 0.3 (median of 30 runs, shaded band is min to max).

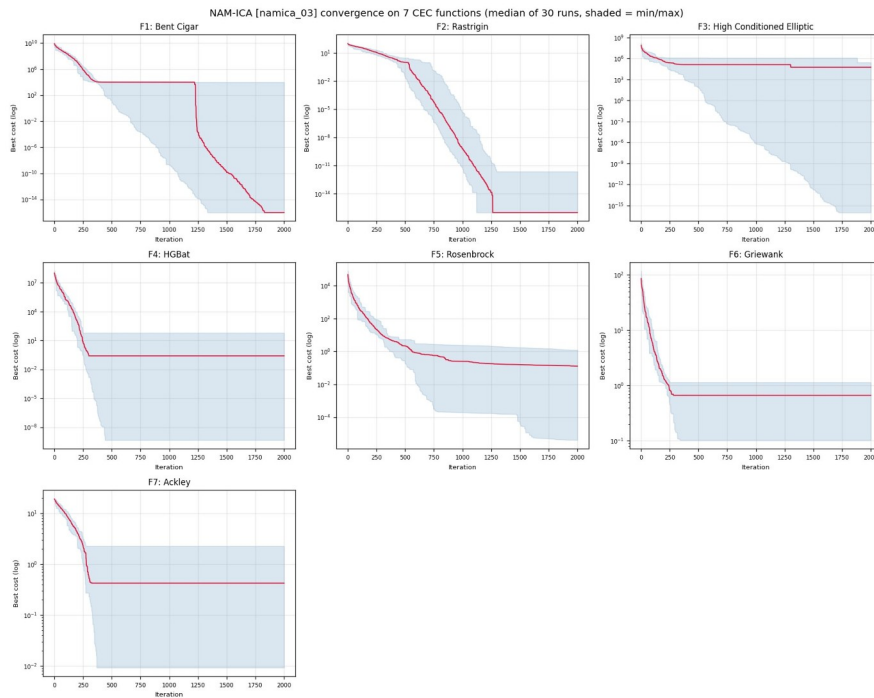


Figure 4. NAM-ICA convergence at revolution 0.3 (median of 30 runs, shaded band is min to max).

What happened on each function

F1 and F2. Already solved by every version, with costs driven down to essentially zero. No room to improve and nothing breaks.

F3 Elliptic. Stuck at the same large value across versions, because its difficulty is the lopsided shape, not false dips. Not a target, and unchanged as expected.

F5 Rosenbrock. A small cost. NAM-ICA at 0.3 is slightly worse than ICA at 0.3, 0.13 against 0.07. Holding empires apart slows the final polishing on a single valley. The difference is not significant.

F4 HGBat. At revolution 0.3 NAM-ICA is slightly worse than ICA, 0.26 against 0.14, and not significant. The interesting result is at revolution 0.5, and it was not planned. There the extra randomness hurts plain ICA, whose median rises to 1.25. NAM-ICA at 0.5 stays at 0.016, and the number of runs reaching a good answer, below 0.1, rose from 10 to 22 out of 30. The Wilcoxon p-value is 0.013, below 0.05, so the difference is significant. This is a secondary observation found after seeing the numbers, not an original aim. It says NAM-ICA behaves better than plain ICA when randomness is high. A small tail of hard runs remained in both versions, so it is steadier, not fixed.

F6 Griewank. No change. All four versions land near 0.6 and the test finds no difference. F6 was a target, and the change did not help here. The likely reason is the shape: Griewank's wide bowl already pulls every empire toward the centre, so changing which empires compete had little to grip. This is a likely explanation, not a proven one.

F7 Ackley. The main positive result. At revolution 0.3 the median drops from 1.23 to 0.42, a large improvement, and the number of runs below 1.0 rose from 14 to 22 out of 30. The Wilcoxon p-value is 0.052, just above 0.05, so by the strict rule it is not significant. Figure 2 shows the limit. Both versions flatten near iteration 300, which means the search reached the right region and then stalled before the very bottom, with NAM-ICA stalling lower than ICA. The limit is premature stalling, not too few iterations, so running longer would not close the gap.

Across the 14 comparisons, one is significant: HGBat at revolution 0.5, the secondary finding. Ackley at revolution 0.3 is a near miss at $p = 0.052$. The rest are ties or small non-significant differences, including the small costs on F4 and F5 at revolution 0.3.

8. Conclusion

One simple structural change, with no new parameters, produced a promising median improvement on the hardest deceptive function, Ackley, while narrowly missing statistical significance. Other results roughly matched plain ICA. The change is fair by construction, since switching it off reproduces plain ICA exactly. The prediction was half right: Ackley improved, Griewank did not. The HGBat result under high randomness was a useful surprise rather than a planned aim. This is a modest result, not a breakthrough.

NAM-ICA adds no experimentally tuned parameter. It uses one fixed design choice: competition starts with the two nearest empires and expands linearly to global competition.

9. Next steps

These follow from the results and are untested.

- Add a local refinement step, a short local search that nudges the best point to find a lower nearby one, to push near-misses like Ackley closer to zero. This would add parameters and extra cost-function calls, which needs to be counted in the budget.
- Test at higher dimensions, for example 30, to see whether the benefit grows or shrinks as the problems get harder.
- Run more than 30 seeds, to confirm the results are stable.
- Start the empires as real spatial clusters instead of a uniform random spread, so the regional separation the change relies on exists from the start.
- Try other shapes for how the competition neighbourhood grows, since the straight-line growth used here was a fixed choice and not tuned.

10. Rejected design variants

NAM-ICA was the last idea standing after a long design phase and the others are worth a quick farewell:

Republic emancipation: split an oversized empire into a new sovereign one.

Power handicap: hard ceiling on empire size, progressive tax on strength

Cultural revolution: periodic full hierarchy reset.

Gulag labor: worst 10% of colonies forced to revolution rate 1.0.

Gagarin scouting: imperialists deliberately target least-visited regions.

Prisoner promotion: a teleported colony becomes a new imperialist.

Samizdat fitness information: high-surprise imperialists nudge all others' coordinates.

Solidarity economy: horizontal colony-to-colony fitness sharing across empires

Migrating empire: a near-collapse empire teleports to new territory

Council of equals: assimilate toward a top-k council centroid instead of one imperialist.

Mutual aid: a strong empire donates colonies to a collapsing one

General strike: global stagnation triggers one synchronized high-revolution round.

Standard toolbox (fillers): chaos maps (logistic, tent), quasi-reflective / opposition-based learning, local search (hill-climbing, firefly), hybridization.

References

Atashpaz-Gargari, E., & Lucas, C. (2007). Imperialist competitive algorithm: An algorithm for optimization inspired by imperialistic competition. In 2007 IEEE Congress on Evolutionary Computation (CEC) (pp. 4661–4667). IEEE.
<https://doi.org/10.1109/CEC.2007.4425083>

Atashpaz-Gargari, E. (2008). Imperialist Competitive Algorithm (ICA) [MATLAB code]. MATLAB Central File Exchange. Retrieved June 11, 2026, from
<https://www.mathworks.com/matlabcentral/fileexchange/22046-imperialist-competitive-algorithm-ica>

Lin, J.-L., Tsai, Y.-H., & Yu, C.-Y. (2012). Interaction enhanced imperialist competitive algorithms. *Algorithms*, 5(4), 433–448. <https://doi.org/10.3390/a5040433>

Talebi, H., Nikoo, H., Mirzaei, A., & Boushehri, S. A. S. (2013). Niching Imperialistic Competitive Algorithm. Conference paper. Available from ResearchGate:
https://www.researchgate.net/publication/255979037_Niching_Imperialistic_Competitive_Algorithm

Soft Computing Optimization Techniques. (2025/2026). Enhancing Swarm Intelligence Algorithms with Local Search on CEC Benchmark Functions [Course assignment]. Singidunum University.